
REQUIREMENTS NOT MET

None

PROBLEMS ENCOUNTERED

None Encountered

FUTURE WORK/APPLICATIONS

ADC is what allows almost any peripheral to work in a quantitative measure. With ADC, we can read the values from photoresistors, waveform inputs, and any other sort of sensor connected to an ADC input pin. Fundamental to creating useful PCBs and products 😊.

PRE-LAB EXERCISES

- i. Why must we use the ADCA module as opposed to the ADCB module?

PortA is connected to the photoresistors pins on PortA, where ADCB is connected to PortB.

- ii. Would it be possible to use any other ADC configurations such as single-ended, differential, differential with gain, etc. with the current pinout and connections of the OOTB Analog Backpack? Why or why not?

Single-ended mode would not be ideal as we want the differential voltage across CDS- and CDS+.

So, Differential is the ideal setting since we want our ADC output to be based on the difference between the CDS- and CDS+ signals. Since the voltage difference is large enough to do without amplification, we do not need gain, so we COULD use gain so long as the gain is small enough to not go higher or lower than the maximum rail limits.

- iii. What would the main benefit be for using a 12-bit ADC system rather than an 8-bit ADC system? Would there be any reason to use 8-bit resolution instead of 12-bit resolution?

A higher bit count means higher resolution, allowing you to be more accurate (i.e, 5.75% battery left instead of 6%)

8-bit would give 256 values, 12-bit would give 4096 values.

Less bits means faster processing and less memory.

- iv. What is the decimal voltage value that is equivalent to a 12-bit signed result of 0x304, given a voltage range of -5V to +5V?

12-bit = 4096 values, with a voltage range between -5V and 5V, we have a 10V range, so our quantization increment will be $10/4096 = 0.002441\text{V}$ per count; $0x304 = 772$, so $772 * 10/4096 \approx 1.8848\text{V}$

- v. Given an 8-bit signed ADC system with a voltage reference range of -3V to +6V, express the expected digital value in terms of the analog input voltage, using the form $V_D = f(V_A)$.

We want -128 at $V_A \leq V_{min}$, and +127 when $V_A \Rightarrow V_{max}$, so to get this we would need our denominator to be $V_{max} - V_{min} = 9$, and our numerator to be $V_A - V_{min}$ (to get 0/9).

This would give us $((V_A + 3)/9) * 255 - 128$, at $V_A = -3$, $0/9 * 255 - 128 = -128$, at $V_A = 6$, $1 * 255 - 128 = 127$.

PSEUDOCODE/FLOWCHARTS

Lab7_1.c

Int main(void){

Create adc_output variable

Adc_init

While(1){

Start ADC

Wait for finish

Store value and reset

}

}

Void adc_init(void){

Set 2.5V ref

Put into a differential with gain mode for pin access (1x gain)

Set resolution to 12bit, right adjusted

Set prescaler

Select positivity/negativity of pins

Enable ADC

}

Lab7_2.c

Int main(){

Create adc_output variable

Set blue/green PWM to outputs

Set blue, clear green

Tcc0_init

Adc_init

Sei

While(1){

// empty loop

}

}

Void tcc0_init(void){

Set period and prescaler for .1 second length

Enable PMIC

Set event system to run off of TCC0 OVF

}

Void adc_init(void){

Set 2.5V ref

Put into a differential with gain mode for pin access (1x gain)

Set resolution to 12bit, right adjusted

Set prescaler

Select positivity/negativity of pins

Set event system control to channel 0, convert on channel 0

Enable ADC

}

ISR(ADCA_CHANNEL0){

Update adc_output

Toggle LEDs

}

Lab7_3.c

Int main{

Tcc0_init

Adc_init

Usartd0_init

Sei

While(1){

Turn off flag

Save adc output to secondary variable so data does not change during loop

Determine sign and save to variable

Calculate decimals from formula on lab doc

Use outchar and outstring to output decimal and format

Use a table with hexchars and index via nibble to output hex values

Carriage return/newline

}}

Void usartd0_init(void){

Set TxD to high, output

Set RxD to input

Set Baud rate via BSEL/BSCALE

Set as asynchronous, odd parity, 8bit channel

Enable Tx/Rx

}

Void outchar(char){

Wait for transfer

Send char to data bus

}

Void outstring(string){

For loop outchar

}

Reuse adc_init, tcc0_init

Update ADC_CH0 ISR to save the output to a variable and set a variable flag.

Lab7_4.c

Main{

Tcc0_init

Adc_init

Usartd0_init

Sei

While (1){

If adc_flag{

Reset flag

Store adc output to secondary variable

Output low byte and high byte using outchar

}}

Reuse usartd0_init, tcc0_init (with 155Hz), adc_init, outchar/outstring, and ISR.

Lab7_5.c

Main{

Tcc0_init

Adc_init

Usartd0_init

Sei

While (1){

If adc_flag{

Reset flag

Store adc output to secondary variable

Output low byte and high byte using outchar

}}}

ISR(usartd0_rxif){

If usartd0.data = 'P'{

Connect photocell pins

}

elseif usartd0.data = 'W'{

Connect AIN pins

}

}

Reuse usartd0_init with receive interrupts, tcc0_init (with 155Hz), adc_init, outchar/outstring, and ISR

PROGRAM CODE

```
Lab7_1.c
/*
 * lab7_1.c
 *
 * Created: 7/19/2026 11:07:50 PM
 * Author : Evan Baesler
 */

#include <avr/io.h>

void adc_init(void);

int main(void){

    int16_t adc_output;

    adc_init();

    while(1) {
        // Start conversion on CH0
        ADCA.CH0.CTRL |= ADC_CH_START_bm;

        // Wait for conversion to finish (check interrupt flag)
        while(!(ADCA.CH0.INTFLAGS & ADC_CH_CHIF_bm));

        // Store result and clear flag
        adc_output = ADCA.CH0.RES;
        ADCA.CH0.INTFLAGS = ADC_CH_CHIF_bm;
    }
}

void adc_init(void) {
    // Set 2.5V voltage reference on PB
    ADCA.REFCTRL = ADC_REFSEL_AREFB_gc;

    // 12-bit signed ADC, right-adjusted with signed mode conversion
    ADCA.CTRLB = ADC_CONMODE_bm | ADC_RESOLUTION_12BIT_gc;

    // Set clock prescaler
    ADCA.PRESCALER = ADC_PRESCALER_DIV512_gc;

    // Select positive and negative inputs (Use DIFF w/ 1x Gain for pin support)
    // Pin1 = CDS+, Pin6 = CDS-
    ADCA.CH0.MUXCTRL = ADC_CH_MUXPOS_PIN1_gc | ADC_CH_MUXNEG_PIN6_gc;

    // Set gain & enable the channel / module
    ADCA.CH0.CTRL = ADC_CH_GAIN_1X_gc;
    ADCA.CH0.CTRL |= ADC_CH_INPUTMODE_DIFFWGAIN_gc;
    ADCA.CTRLA = ADC_ENABLE_bm;
}
```

Lab7_2.c

```
/*
 * lab7_2.c
 *
 * Created: 7/20/2026 6:38:19 PM
 * Author: Evan Baesler
 */

#include <avr/io.h>
#include <avr/interrupt.h>

int16_t adc_output = 0;

void tcc0_init(void);
void adc_init(void);

int main(void){

    PORTD.DIRSET = PIN5_bm | PIN6_bm;

    PORTD.OUTSET = PIN5_bm;
    PORTD.OUTCLR = PIN6_bm;

    tcc0_init();
    adc_init();

    sei();

    while(1){

    }

}

void tcc0_init(void){

    // Do NOT put a overflow interrupt, event system handles it
    TCC0.PER = 3125; // .1(2000000/64) = 3125
    TCC0.CTRLA = TC_CLKSEL_DIV64_gc; // .1(2000000/64) = 3125

    PMIC.CTRL = PMIC_LOLVLEN_bm; // Enable low-level interrupts

    EVSYS.CH0MUX = EVSYS_CHMUX_TCC0_OVF_gc; // Send TCC0 OVF to event system
}
```

```
// From lab7_1, expanded

void adc_init(void) {
    // Set 2.5V voltage reference on PB
    ADCA.REFCTRL = ADC_REFSEL_AREFB_gc;

    // 12-bit signed ADC, right-adjusted with signed mode conversion
    ADCA.CTRLB = ADC_CONMODE_bm | ADC_RESOLUTION_12BIT_gc;

    // Set clock prescaler
    ADCA.PRESCALER = ADC_PRESCALER_DIV512_gc;

    // Select positive and negative inputs (Use DIFF w/ 1x Gain for pin support)
    // Pin1 = CDS+, Pin6 = CDS-
    ADCA.CH0.MUXCTRL = ADC_CH_MUXPOS_PIN1_gc | ADC_CH_MUXNEG_PIN6_gc;

    // Set gain & enable the channel / module
    ADCA.CH0.CTRL = ADC_CH_GAIN_1X_gc;
    ADCA.CH0.CTRL |= ADC_CH_INPUTMODE_DIFFWGAIN_gc;

    // Enable ADC low-level interrupt
    ADCA.CH0.INTCTRL = ADC_CH_INTLVL_LO_gc;

    // Allow event system control to use channels 0-3, do conversion on CH0
    ADCA.EVCTRL = ADC_EVSEL_0123_gc | ADC_EVACT_CH0_gc;

    // Enable ADC :)
    ADCA.CTRLA = ADC_ENABLE_bm;
}

ISR(ADCA_CH0_vect){

    // Update ADC Output
    adc_output = ADCA.CH0.RES;

    // Toggle LEDs
    PORTD.OUTTGL = PIN5_bm | PIN6_bm;

}
```

Lab7_3.c

```
/*
 * lab7_3.c
 *
 * Created: 7/20/2026 9:45:32 PM
 * Author: Evan Baessler
 */

#include <avr/io.h>
#include <avr/interrupt.h>

int8_t BSEL = 17;
int8_t BSCALE = -7; // 110400 bps
int16_t adc_output = 0;
volatile uint8_t adc_flag = 0;

void tcc0_init(void);
void adc_init(void);
void usartd0_init(void);
void outchar(char c);
void outstring(const char *str);

int main(void){

    tcc0_init();
    adc_init();
    usartd0_init();

    sei();

    while(1){
        if(adc_flag){
            adc_flag = 0;

            int16_t data = adc_output;
            char sign = '+';

            if (data < 0){
                sign = '-';
                data = -data; // Remove two's comp so we can make it easier to read.
            }

            float voltage = ((float)data / 2048.0f) * 2.5f;
            int int1 = voltage;
            float voltage2 = 10.0f * (voltage - int1);
            int int2 = voltage2;
            float voltage3 = 10.0f * (voltage2 - int2);
            int int3 = voltage3;

            outstring("Transmitted Result: ");
            outchar(sign);
            outchar('0' + int1); // ASCII 0 + int lets us output a number
            outchar('.');
            outchar('0' + int2);
            outchar('0' + int3);
            outstring(" V (0x)");

            // Output 3-digit Hex value
            char hex_chars[] = "0123456789ABCDEF"; // Treats as an array of hex_chars[i]
```

```
        outchar(hex_chars[(data >> 8) & 0x0F]); // Shifts hex to the right, and uses the
byte value as
        outchar(hex_chars[(data >> 4) & 0x0F]); // a way to access hex_chars[i].
        outchar(hex_chars[data & 0x0F]);

        outstring("\r\n");
    }
}

void tcc0_init(void){
    // Do NOT put a overflow interrupt, event system handles it
    TCC0.PER = 3125; // .1(2000000/64) = 3125
    TCC0.CTRLA = TC_CLKSEL_DIV64_gc; // .1(2000000/64) = 3125

    PMIC.CTRL = PMIC_LOLVLEN_bm; // Enable low-level interrupts

    EVSYS.CH0MUX = EVSYS_CHMUX_TCC0_OVF_gc; // Send TCC0 OVF to event system
}

// From lab7_1

void adc_init(void) {
    // Set 2.5V voltage reference on PB
    ADCA.REFCTRL = ADC_REFSEL_AREFB_gc;

    // 12-bit signed ADC, right-adjusted with signed mode conversion
    ADCA.CTRLB = ADC_CONMODE_bm | ADC_RESOLUTION_12BIT_gc;

    // Set clock prescaler
    ADCA.PRESCALER = ADC_PRESCALER_DIV512_gc;

    // Select positive and negative inputs (Use DIFF w/ 1x Gain for pin support)
    // Pin1 = CDS+, Pin6 = CDS-
    ADCA.CH0.MUXCTRL = ADC_CH_MUXPOS_PIN1_gc | ADC_CH_MUXNEG_PIN6_gc;

    // Set gain & enable the channel / module
    ADCA.CH0.CTRL = ADC_CH_GAIN_1X_gc;
    ADCA.CH0.CTRL |= ADC_CH_INPUTMODE_DIFFWGAIN_gc;

    // Enable ADC low-level interrupt
    ADCA.CH0.INTCTRL = ADC_CH_INTLVL_LO_gc;

    // Allow event system control to use channels 0-3, do conversion on CH0
    ADCA.EVCTRL = ADC_EVSEL_0123_gc | ADC_EVACT_CH0_gc;

    // Enable ADC :)
    ADCA.CTRLA = ADC_ENABLE_bm;
}
}
```

```
void usartd0_init(void){
    PORTD.OUTSET = PIN3_bm; // TxD set high
    PORTD.DIRSET = PIN3_bm; // TxD output
    PORTD.DIRCLR = PIN4_bm; // RxD input

    USARTD0.BAUDCTRLA = BSEL;
    USARTD0.BAUDCTRLB = (BSCALE << 4);

    // Asynchronous, 8 data bits, ODD parity, 1 stop bit
    USARTD0.CTRLA = USART_CMODE_ASYNCHRONOUS_gc | USART_PMODE_ODD_gc | USART_CHSIZE_8BIT_gc;

    USARTD0.CTRLB = USART_TXEN_bm | USART_RXEN_bm;
}

void outchar(char c){
    while(!(USARTD0.STATUS & USART_DREIF_bm));
    USARTD0.DATA = c;
}

void outstring(const char *str){
    for(uint8_t i = 0; str[i] != 0; i++){
        outchar(str[i]);
    }
}

ISR(ADCA_CH0_vect){

    // Update ADC Output
    adc_output = ADCA.CH0.RES;
    adc_flag = 1;
}
```

Lab7_4.c

```
/*  
  
 * lab7_4.c  
 *  
 * Created: 7/21/2026 9:47:41 AM  
 * Author: Evan Baesler  
 */  
  
#include <avr/io.h>  
#include <avr/interrupt.h>  
  
int8_t BSEL = 17;  
int8_t BSCALE = -7; // 110400 bps  
int16_t adc_output = 0;  
volatile uint8_t adc_flag = 0;  
  
void tcc0_init(void);  
void adc_init(void);  
void usartd0_init(void);  
void outchar(char c);  
void outstring(const char *str);  
  
int main(void){  
  
    tcc0_init();  
    adc_init();  
    usartd0_init();  
  
    sei();  
  
    while(1){  
if(adc_flag){  
    adc_flag = 0;  
  
    int16_t data = adc_output;  
  
    // Send low byte, then high byte (or vice-versa depending on endianness)  
    outchar(data & 0xFF); // Low byte  
    outchar((data >> 8) & 0xFF); // High byte  
    }  
    }  
}  
  
void tcc0_init(void){  
  
    // Do NOT put a overflow interrupt, event system handles it  
    TCC0.PER = 1612; // (1/155)(2000000/8) ~= 1612  
    TCC0.CTRLA = TC_CLKSEL_DIV8_gc; // (1/155)(2000000/8) ~= 1612  
  
    PMIC.CTRL = PMIC_LOLVLEN_bm; // Enable low-level interrupts  
  
    EVSYS.CH0MUX = EVSYS_CHMUX_TCC0_OVF_gc; // Send TCC0 OVF to event system  
}
```

```
// From lab7_1
void adc_init(void) {
    // Set 2.5V voltage reference on PB
    ADCA.REFCTRL = ADC_REFSEL_AREFB_gc;

    // 12-bit signed ADC, right-adjusted with signed mode conversion
    ADCA.CTRLB = ADC_CONMODE_bm | ADC_RESOLUTION_12BIT_gc;

    // Set clock prescaler
    ADCA.PRESCALER = ADC_PRESCALER_DIV512_gc;

    // Select positive and negative inputs (Use DIFF w/ 1x Gain for pin support)
    // Pin1 = CDS+, Pin6 = CDS-
    ADCA.CH0.MUXCTRL = ADC_CH_MUXPOS_PIN1_gc | ADC_CH_MUXNEG_PIN6_gc;

    // Set gain & enable the channel / module
    ADCA.CH0.CTRL = ADC_CH_GAIN_1X_gc;
    ADCA.CH0.CTRL |= ADC_CH_INPUTMODE_DIFFWGAIN_gc;

    // Enable ADC low-level interrupt
    ADCA.CH0.INTCTRL = ADC_CH_INTLVL_LO_gc;

    // Allow event system control to use channels 0-3, do conversion on CH0
    ADCA.EVCTRL = ADC_EVSEL_0123_gc | ADC_EVACT_CH0_gc;

    // Enable ADC :)
    ADCA.CTRLA = ADC_ENABLE_bm;
}

void usartd0_init(void){
    PORTD.OUTSET = PIN3_bm; // TxD set high
    PORTD.DIRSET = PIN3_bm; // TxD output
    PORTD.DIRCLR = PIN4_bm; // RxD input

    USARTD0.BAUDCTRLA = BSEL;
    USARTD0.BAUDCTRLB = (BSCALE << 4);

    // Asynchronous, 8 data bits, ODD parity, 1 stop bit
    USARTD0.CTRLC = USART_CMODE_ASYNCHRONOUS_gc | USART_PMODE_ODD_gc | USART_CHSIZE_8BIT_gc;

    USARTD0.CTRLB = USART_TXEN_bm | USART_RXEN_bm;
}

void outchar(char c){
    while(!(USARTD0.STATUS & USART_DREIF_bm));
    USARTD0.DATA = c;
}

void outstring(const char *str){
    for(uint8_t i = 0; str[i] != 0; i++){
        outchar(str[i]);
    }
}

ISR(ADCA_CH0_vect){
    // Update ADC Output
    adc_output = ADCA.CH0.RES;
    adc_flag = 1;
}
```

Lab7_5.c

```
/*
 * lab7_5.c
 *
 * Created: 7/21/2026 6:21:19 PM
 * Author: Evan Baesler
 */

#include <avr/io.h>
#include <avr/interrupt.h>

int8_t BSEL = 17;
int8_t BSCALE = -7; // 110400 bps
int16_t adc_output = 0;
volatile uint8_t adc_flag = 0;
char usart_input = 0;

void tcc0_init(void);
void adc_init(void);
void usartd0_init(void);
void outchar(char c);
void outstring(const char *str);

int main(void){

    tcc0_init();
    adc_init();
    usartd0_init();

    sei();

    while(1){
        if(adc_flag){
            adc_flag = 0;

            int16_t data = adc_output;

            // Send low byte, then high byte (or vice-versa depending on endianness)
            outchar(data & 0xFF); // Low byte
            outchar((data >> 8) & 0xFF); // High byte
        }
    }
}

void tcc0_init(void){

    // Do NOT put a overflow interrupt, event system handles it
    TCC0.PER = 1612; // (1/155)(2000000/8) ~= 1612
    TCC0.CTRLA = TC_CLKSEL_DIV8_gc; // (1/155)(2000000/8) ~= 1612

    PMIC.CTRL = PMIC_LOLVLEN_bm; // Enable low-level interrupts

    EVSYS.CH0MUX = EVSYS_CHMUX_TCC0_OVF_gc; // Send TCC0 OVF to event system
}

// From lab7_1

void adc_init(void) {
    // Set 2.5V voltage reference on PB
    ADCA.REFCTRL = ADC_REFSEL_AREFB_gc;
}
```

```
// 12-bit signed ADC, right-adjusted with signed mode conversion
ADCA.CTRLB = ADC_CONMODE_bm | ADC_RESOLUTION_12BIT_gc;

// Set clock prescaler
ADCA.PRESCALER = ADC_PRESCALER_DIV512_gc;

// Select positive and negative inputs (Use DIFF w/ 1x Gain for pin support)
// Pin1 = CDS+, Pin6 = CDS-
ADCA.CH0.MUXCTRL = ADC_CH_MUXPOS_PIN1_gc | ADC_CH_MUXNEG_PIN6_gc;

// Set gain & enable the channel / module
ADCA.CH0.CTRL = ADC_CH_GAIN_1X_gc;
ADCA.CH0.CTRL |= ADC_CH_INPUTMODE_DIFFWGAIN_gc;

// Enable ADC low-level interrupt
ADCA.CH0.INTCTRL = ADC_CH_INTLVL_LO_gc;

// Allow event system control to use channels 0-3, do conversion on CH0
ADCA.EVCTRL = ADC_EVSEL_0123_gc | ADC_EVACT_CH0_gc;

// Enable ADC :)
ADCA.CTRLA = ADC_ENABLE_bm;

}

void usartd0_init(void){
    PORTD.OUTSET = PIN3_bm; // TxD set high
    PORTD.DIRSET = PIN3_bm; // TxD output
    PORTD.DIRCLR = PIN4_bm; // RxD input

    USARTD0.BAUDCTRLA = BSEL;
    USARTD0.BAUDCTRLB = (BSCALE << 4);

    // Asynchronous, 8 data bits, ODD parity, 1 stop bit
    USARTD0.CTRLA = USART_CMODE_ASYNCHRONOUS_gc | USART_PMODE_ODD_gc | USART_CHSIZE_8BIT_gc;

    USARTD0.CTRLB = USART_TXEN_bm | USART_RXEN_bm;
    USARTD0.CTRLA = USART_RXCINTLVL_LO_gc;
}

void outchar(char c){
    while(!(USARTD0.STATUS & USART_DREIF_bm));
    USARTD0.DATA = c;
}

void outstring(const char *str){
    for(uint8_t i = 0; str[i] != 0; i++){
        outchar(str[i]);
    }
}

ISR(ADCA_CH0_vect){

    // Update ADC Output
    adc_output = ADCA.CH0.RES;
    adc_flag = 1;

}

ISR(USARTD0_RXC_vect){
```

```
    usart_input = USARTD0.DATA;

    if (usart_input == ('P' | 'p')){

        ADCA.CH0.MUXCTRL = ADC_CH_MUXPOS_PIN1_gc | ADC_CH_MUXNEG_PIN6_gc;

    }

    else if (usart_input == ('W' | 'w')){

        ADCA.CH0.MUXCTRL = ADC_CH_MUXPOS_PIN5_gc | ADC_CH_MUXNEG_PIN4_gc;

    }

}
```

APPENDIX

Word Address			
ATxmega128A1U		ATxmega64A1U	
0		0	Application Section (bytes) (128K/64K)
			...
FFFF	/	77FF	Application Table Section (bytes) (8K/ 4K)
F000	/	7800	
FFFF	/	7FFF	
10000	/	8000	Boot Section (bytes) (8K/4K)
10FFF	/	87FF	

Figure 1: Program Memory Addressing

Byte Address	ATxmega64A1U	Byte Address	ATxmega128A1U
0	I/O Registers (4K)	0	I/O Registers (4KB)
FFF		FFF	
1000	EEPROM (2K)	1000	EEPROM (2K)
17FF		17FF	
	RESERVED		RESERVED
2000	Internal SRAM (4K)	2000	Internal SRAM (8K)
2FFF		3FFF	
4000	External Memory (0 to 16MB)	3000	External Memory (0 to 16MB)
FFFFFF		FFFFFF	

Figure 2: Data Memory Addressing

13.16 Register summary – Ports

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page
+0x00	DIR	DIR[7:0]								148
+0x01	DIRSET	DIRSET[7:0]								148
+0x02	DIRCLR	DIRCLR[7:0]								148
+0x03	DIRTGL	DIRTGL[7:0]								148
+0x04	OUT	OUT[7:0]								149
+0x05	OUTSET	OUTSET[7:0]								149
+0x06	OUTCLR	OUTCLR[7:0]								149
+0x07	OUTTGL	OUTTGL[7:0]								149
+0x08	IN	IN[7:0]								150
+0x09	INTCTRL	–	–	–	–	INT1LVL[1:0]		INT0LVL[1:0]		150
+0x0A	INT0MASK	INT0MSK[7:0]								150
+0x0B	INT1MASK	INT1MSK[7:0]								150
+0x0C	INTFLAGS	–	–	–	–	–	–	INT1IF	INT0IF	151
+0x0D	Reserved	–	–	–	–	–	–	–	–	
+0x0E	REMAP	–	–	SPI	USART0	TC0D	TC0C	TC0B	TC0A	151
+0x0F	Reserved	–	–	–	–	–	–	–	–	
+0x10	PIN0CTRL	SRLEN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x11	PIN1CTRL	SRLEN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x12	PIN2CTRL	SRLEN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x13	PIN3CTRL	SRLEN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x14	PIN4CTRL	SRLEN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x15	PIN5CTRL	SRLEN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x16	PIN6CTRL	SRLEN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x17	PIN7CTRL	SRLEN	INVEN	OPC[2:0]			ISC[2:0]			152
+0x18	Reserved	–	–	–	–	–	–	–	–	
+0x19	Reserved	–	–	–	–	–	–	–	–	
+0x1A	Reserved	–	–	–	–	–	–	–	–	
+0x1B	Reserved	–	–	–	–	–	–	–	–	
+0x1C	Reserved	–	–	–	–	–	–	–	–	
+0x1D	Reserved	–	–	–	–	–	–	–	–	
+0x1E	Reserved	–	–	–	–	–	–	–	–	
+0x1F	Reserved	–	–	–	–	–	–	–	–	

Figure 3: Ports Register Summary

14.13 Register summary

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page
+0x00	CTRLA	–	–	–	–	CLKSEL[3:0]				175
+0x01	CTRLB	CCDEN	CCCEN	CCBEN	CCAEN	–	WGMD[2:0]			175
+0x02	CTRLC	–	–	–	–	CMPD	CMPC	CMPB	CMPA	176
+0x03	CTRLD	EVACT[2:0]			EVDLY	EVSEL[3:0]				176
+0x04	CTRLF	–	–	–	–	–	–	BYTEM		178
+0x05	Reserved	–	–	–	–	–	–	–	–	
+0x06	INTCTRLA	–	–	–	–	ERRINTLV[1:0]		OVINTLV[1:0]		178
+0x07	INTCTRLB	CCCINTLV[1:0]		CCBINTLV[1:0]		CCAVINTLV[1:0]		CCOINTLV[1:0]		178
+0x08	CTRLCLR	–	–	–	QDECINDX	CMD[1:0]		LUPD	DIR	179
+0x09	CTRLFSET	–	–	–	QDECINDX	CMD[1:0]		LUPD	DIR	180
+0x0A	CTRLGCLR	–	–	–	CCDBV	CCCBV	CCBBV	CCABV	PERBV	180
+0x0B	CTRLGSET	–	–	–	CCDBV	CCCBV	CCBBV	CCABV	PERBV	180
+0x0C	INTFLAGS	CCDIF	CCCIF	CCBIF	CCAIF	–	–	ERRIF	OVFIF	180
+0x0D	Reserved	–	–	–	–	–	–	–	–	
+0x0E	Reserved	–	–	–	–	–	–	–	–	
+0x0F	TEMP	TEMP[7:0]								181
+0x10 to +0x1F	Reserved	–	–	–	–	–	–	–	–	
+0x20	CNTL	CNT[7:0]								181
+0x21	CNTH	CNT[15:8]								181
+0x22 to +0x25	Reserved	–	–	–	–	–	–	–	–	
+0x26	PERL	PER[7:0]								182
+0x27	PERH	PER[8:15]								182
+0x28	CCAL	CCA[7:0]								182
+0x29	CAAH	CCA[15:8]								182
+0x2A	CCBL	CCB[7:0]								182
+0x2B	CBH	CCB[15:8]								182
+0x2C	CCCL	CCC[7:0]								182
+0x2D	CCCH	CCC[15:8]								182
+0x2E	CCDL	CCD[7:0]								182
+0x2F	CCDH	CCD[15:8]								182
+0x30 to +0x35	Reserved	–	–	–	–	–	–	–	–	
+0x36	PERBUFL	PERBUF[7:0]								183

Figure 4: Timer/Counter Register Summary

12.9 Register summary

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page
+0x00	STATUS	NMIEX	–	–	–	–	HILVLEX	MEDLVLEX	LOLVLEX	136
+0x01	INTPRI	INTPRI[7:0]								136
+0x02	CTRL	RREN	IVSEL	–	–	–	HILVLEN	MEDLVLEN	LOLVLEN	137

Figure 5: Interrupts Register Summary

27.12 Register summary – EBI

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page	
+0x00	CTRL	SDDATAW[1:0]		LPCMODE[1:0]		SRMODE[1:0]		IFMODE[1:0]		329	
+0x01	SDRAMCTRLA	—	—	—	—	SDCAS	SDROW	SDCOL[1:0]		330	
+0x02	Reserved	—	—	—	—	—	—	—	—		
+0x03	Reserved	—	—	—	—	—	—	—	—		
+0x04	REFRESHL	SDRAM Refresh Period Low Byte								331	
+0x05	REFRESHH	—	—	—	—	—	—	SDRAM Refresh Period High		331	
+0x06	INITDLYL	SDRAM Initialization Time Low Byte								331	
+0x07	INITDLYH	—	—	SDRAM Initialization Time High Byte							331
+0x08	SDRAMCTRLB	MRDLY[1:0]		ROWCYCDLY[[2:0]			RPDLY[2:0]			332	
+0x09	SDRAMCTRLC	WRDLY[1:0]		ESRDLY[2:0]			ROWCOLDLY[2:0]			333	
+0x0A	Reserved	—	—	—	—	—	—	—	—		
+0x0B	Reserved	—	—	—	—	—	—	—	—		
+0x0C	Reserved	—	—	—	—	—	—	—	—		
+0x0D	Reserved	—	—	—	—	—	—	—	—		
+0x0E	Reserved	—	—	—	—	—	—	—	—		
+0x0F	Reserved	—	—	—	—	—	—	—	—		
+0x10	CS0	Chip Select 0 Offset Address									
+0x14	CS1	Chip Select 1 Offset Address									
+0x18	CS2	Chip Select 2 Offset Address									
+0x1C	CS3	Chip Select 3 Offset Address									

Figure 6: EBI Register Summary

27.13 Register summary – EBI chip select

Address	Name		Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page	
+0x00	CTRLA		–	ASIZE[4:0]					MODE[1:0]		329	
+0x01	CTRLB	(SRAM)	–	–	–	–	–	SRWS[2:0]			330	
		(SDRAM)	SDINITDONE	–	–	–	–	SDSREN	SDMODE[1:0]			
+0x02	BASEADDRL		Chip Select Base Address Low Byte					–	–	–	–	337
+0x03	BASEADDRH		Chip Select Base Address High Byte									337

Figure 7: EBI Chip Select Register Summary

Table 33-7. Port H - alternate functions.

PORT H	PIN #	INTERRUPT	SDRAM 3P	SRAM ALE1	SRAM ALE12	LPC3 ALE1	LPC2 ALE1	LPC2 ALE12
GND	53							
VCC	54							
PH0	55	SYNC	$\overline{WE}$	$\overline{WE}$	$\overline{WE}$	$\overline{WE}$	$\overline{WE}$	$\overline{WE}$
PH1	56	SYNC	$\overline{CAS}$	$\overline{RE}$	$\overline{RE}$	$\overline{RE}$	$\overline{RE}$	$\overline{RE}$
PH2	57	SYNC/ASYN	$\overline{RAS}$	$\overline{ALE1}$	$\overline{ALE1}$	$\overline{ALE1}$	$\overline{ALE1}$	$\overline{ALE1}$
PH3	58	SYNC	$\overline{DQM}$		$\overline{ALE2}$			$\overline{ALE2}$
PH4	59	SYNC	BA0	$\overline{CS0}/A16$	$\overline{CS0}$	$\overline{CS0}/A16$	$\overline{CS0}$	$\overline{CS0}/A16$
PH5	60	SYNC	BA1	$\overline{CS1}/A17$	$\overline{CS1}$	$\overline{CS1}/A17$	$\overline{CS1}$	$\overline{CS1}/A17$
PH6	61	SYNC	CKE	$\overline{CS2}/A18$	$\overline{CS2}$	$\overline{CS2}/A18$	$\overline{CS2}$	$\overline{CS2}/A18$
PH7	62	SYNC	CLK	$\overline{CS3}/A19$	$\overline{CS3}$	$\overline{CS3}/A19$	$\overline{CS3}$	$\overline{CS3}/A19$

Notes: 1. CS0 - CS3 can optionally be moved to Port E or F
2. A16-A23 can optionally be moved to Port E or F when EBI configured in 4PORT mode. Refer to the EBIOUT register description in the XMEGA AU Manual.

Figure 8: PORTH Alternate Functions

Table 33-8. Port J - alternate functions.

PORT J	PIN #	INTERRUPT	SDRAM 3P	SRAM ALE1	SRAM ALE12	LPC3 ALE1	LPC2 ALE1	LPC2 ALE12
GND	63							
VCC	64							
PJ0	65	SYNC	D0	D0	D0	D0/A0	D0/A0	D0/A0/A8
PJ1	66	SYNC	D1	D1	D1	D1/A1	D1/A1	D1/A1/A9

ATxmega128A1U/ATxmega64A1U

PORT J	PIN #	INTERRUPT	SDRAM 3P	SRAM ALE1	SRAM ALE12	LPC3 ALE1	LPC2 ALE1	LPC2 ALE12
PJ2	67	SYNC/ASYN	D2	D2	D2	D2/A2	D2/A2	D2/A2/A10
PJ3	68	SYNC	D3	D3	D3	D3/A3	D3/A3	D3/A3/A11
PJ4	69	SYNC	A8	D4	D4	D4/A4	D4/A4	D4/A4/A12
PJ5	70	SYNC	A9	D5	D5	D5/A5	D5/A5	D5/A5/A13
PJ6	71	SYNC	A10	D6	D6	D6/A6	D6/A6	D6/A6/A14
PJ7	72	SYNC	A11	D7	D7	D7/A7	D7/A7	D7/A7/A15

Figure 9: PORTJ Alternate Functions

Table 33-9. Port K - alternate functions.

PORT K	PIN #	INTERRUPT	SDRAM 3P	SRAM ALE1	SRAM ALE2	LPC3 ALE1
GND	73					
VCC	74					
PK0	75	SYNC	A0	A0/A8	A0/A8/A16	A8
PK1	76	SYNC	A1	A1/A9	A1/A9/A17	A9
PK2	77	SYNC/ASYNC	A2	A2/A10	A2/A10/A18	A10
PK3	78	SYNC	A3	A3/A11	A3/A11/A19	A11
PK4	79	SYNC	A4	A4/A12	A4/A12/A20	A12
PK5	80	SYNC	A5	A5/A13	A5/A13/A21	A13
PK6	81	SYNC	A6	A6/A14	A6/A14/A22	A14
PK7	82	SYNC	A7	A7/A15	A7/A15/A23	A15

Figure 10: PORTK Alternate Functions

23.16 Register summary

23.16.1 Register description – USART

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page
+0x00	DATA	DATA[7:0]								294
+0x01	STATUS	RXCIF	TXCIF	DREIF	FERR	BUFOVF	PERR	–	RXB8	294
+0x02	Reserved	–	–	–	–	–	–	–	–	
+0x03	CTRLA	–	–	RXCINTLVL[1:0]		TXCINTLVL[1:0]		DREINTLVL[1:0]		295
+0x04	CTRLB	–	–	–	RXEN	TXEN	CLK2X	MPCM	TXB8	296
+0x05	CTRLC	CMODE[1:0]		PMODE[1:0]		SBMODE	CHSIZE[2:0]			296
+0x06	BAUDCTRLA	BSEL[7:0]								298
+0x07	BAUDCTRLB	BSCALE[3:0]				BSEL[11:8]				298

23.16.2 Register description – USART in SPI Master Mode

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page
+0x00	DATA	DATA[7:0]								294
+0x01	STATUS	RXCIF	TXCIF	DREIF	–	–	–	–	–	294
+0x02	Reserved	–	–	–	–	–	–	–	–	
+0x03	CTRLA	–	–	RXCINTLVL[1:0]		TXCINTLVL[1:0]		DREINTLVL[1:0]		295
+0x04	CTRLB	–	–	–	RXEN	TXEN	–	–	–	296
+0x05	CTRLC	CMODE[1:0]		–	–	–	UDORD	UCPHA	–	296
+0x06	BAUDCTRLA	BSEL[7:0]								298
+0x07	BAUDCTRLB	BSCALE[3:0]				BSEL[11:8]				298

Figure 11: USART Register Summaries

Table 33-3. Port C - alternate functions.

PORT C	PIN#	INTERRUPT	TCC0 ⁽¹⁾⁽²⁾	AWEXC	TCC1	USARTC0 ⁽³⁾	USARTC1	SPIC ⁽⁴⁾	TWIC	CLOCKOUT ⁽⁵⁾	EVENTOUT ⁽⁶⁾
GND	13										
VCC	14										
PC0	15	SYNC	OC0A	OC0ALS					SDA		

ATxmega128A1U/ATxmega64A1U

PORT C	PIN#	INTERRUPT	TCC0 ⁽¹⁾⁽²⁾	AWEXC	TCC1	USARTC0 ⁽³⁾	USARTC1	SPIC ⁽⁴⁾	TWIC	CLOCKOUT ⁽⁵⁾	EVENTOUT ⁽⁶⁾
PC1	16	SYNC	OC0B	OC0AHS		XCK0			SCL		
PC2	17	SYNC/ASYNC	OC0C	OC0BLS		RXD0					
PC3	18	SYNC	OC0D	OC0BHS		TXD0					
PC4	19	SYNC		OC0CLS	OC1A			SS			
PC5	20	SYNC		OC0CHS	OC1B		XCK1	MOSI			
PC6	21	SYNC		OC0DLS			RXD1	MISO		clk _{RTC}	
PC7	22	SYNC		OC0DHS			TXD1	SCK		clk _{PER}	EVOUT

- Notes:
1. Pin mapping of all TC0 can optionally be moved to high nibble of port. Refer to Pin Remap register in I/O Ports in the XMEGA AU Manual.
 2. If TC0 is configured as TC2 all eight pins can be used for PWM output. Refer to Pin Remap register in I/O Ports in the XMEGA AU Manual..
 3. Pin mapping of all USART0 can optionally be moved to high nibble of port. Refer to Pin Remap register in I/O Ports in the XMEGA AU Manual..
 4. Pins MOSI and SCK for all SPI can optionally be swapped. Refer to Pin Remap register in I/O Ports in the XMEGA AU Manual.
 5. CLKOUT can optionally be moved between port C, D and E and between pin 4 and 7. Refer to CLKEVOUT register in I/O port configuration in the XMEGA AU Manual.
 6. EVOUT can optionally be moved between port C, D and E and between pin 4 and 7. Refer to CLKEVOUT register in I/O port configuration in the XMEGA AU Manual.

Figure 12: PORTC Alternate Functions

Notes: 1. All pins on the port can optionally be used for EBI chip select or address lines. Refer to the EBIOUT register description in the XMEGA AU Manual.

Table 33-6. Port F - alternate functions.

PORT F	PIN #	INTERRUPT	TCF0	TCF1	USARTF0	USARTF1	SPIF	TWIF
GND	43							
VCC	44							
PF0	45	SYNC	OC0A					SDA
PF1	46	SYNC	OC0B		XCK0			SCL
PF2	47	SYNC/ASYNC	OC0C		RXD0			
PF3	48	SYNC	OC0D		TXD0			
PF4	49	SYNC		OC1A			$\overline{SS}$	
PF5	50	SYNC		OC1B		XCK1	MOSI	
PF6	51	SYNC				RXD1	MISO	
PF7	52	SYNC				TXD1	SCK	

Note: 1. All pins on the port can optionally be used for EBI chip select or address lines. Refer to the EBIOUT register description in the XMEGA AU Manual.

Figure 13: PORTF Alternate Functions

22.8 Register summary

Address	Name	Bit 7	Bit 6	Bit 5	Bit 4	Bit 3	Bit 2	Bit 1	Bit 0	Page
+0x00	CTRL	CLK2X	ENABLE	DORD	MASTER	MODE[1:0]		PRESCALER[1:0]		276
+0x01	INTCTRL	–	–	–	–	–	–	INTLVL[1:0]		277
+0x02	STATUS	IF	WRCOL	–	–	–	–	–	–	277
+0x03	DATA	DATA[7:0]								278

22.9 Interrupt vector summary

Table 22-4. SPI interrupt vector and its offset word address.

Offset	Source	Interrupt Description
0x00	SPI_vect	SPI interrupt vector

Figure 14: SPI Register Summary

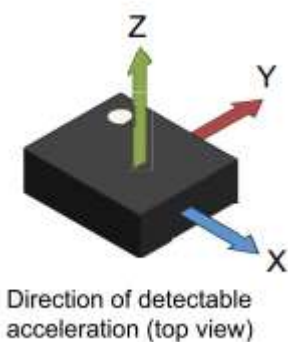


Figure 15: Axis Directions for IMU